Empirical Software Engineering manuscript No.
(will be inserted by the editor)

What Carries Architectural Signal? A Code-to-Architecture Mapping Study

Rakhshanda Jabeen · **Morgan Ericsson** · **Yuyao Duan** · **Anna Wingkvist**

Received: date / Accepted: date

Local review draft, 5 October 2026. Full-dataset results integrated; manuscript and review artifacts remain unpublished.

Abstract Assigning source files to reference architecture modules supports conformance checking and system understanding. Code text, static dependencies, folder organization, and member-level information can inform this mapping, but their contributions depend on how they are represented and used. We investigate these contributions on ten Java, C, and C++ systems with limited labeled data. We compare lexical, structural, and contextual file representations using a multilayer perceptron (MLP), a Graph Attention Network (GAT), and a Heterogeneous Graph Transformer (HGT). With HGT, we vary graph granularity from source files alone to files with their members and enclosing folders. We also control access to dependency-neighbor information within fixed predictors and compare aggregation in one or both directions.

Representations that encode dependencies provide strong baselines when evaluated with the MLP. Additional neighbor information can improve mapping, but its benefit depends on the predictor and aggregation direction; under aggregation in both directions, the GAT achieves the highest median macro-F1 with every representation. Within HGT variants, folder nodes produce the largest and most consistent gains, while member nodes provide no consistent improvement. However, a directory rule using the same labeled files reaches similar accuracy on most systems, folder paths given as file features match or exceed folder nodes on most systems, and folder gains do not consistently extend to files the rule assigns incorrectly. Evaluation should therefore distinguish the contribution of additional graph context from information already available in file representations and directory organization.

Keywords Code-to-Architecture Mapping · Software Architecture · Graph Neural Networks · Heterogeneous Graphs · Representation Learning

Rakhshanda Jabeen
Electrolux Professional AB, Ljungby, Sweden
Linnaeus University, Växjö, Sweden
E-mail: rakhshanda.jabeen@electroluxprofessional.com

Morgan Ericsson
Linnaeus University, Växjö, Sweden
E-mail: morgan.ericsson@lnu.se

Yuyao Duan
Linnaeus University, Växjö, Sweden
E-mail: yuyaoduan4@gmail.com

Anna Wingkvist
Linnaeus University, Växjö, Sweden
E-mail: anna.wingkvist@lnu.se

1 Introduction

Maintaining a software architecture requires knowing which implementation elements belong to its modules. That correspondence can become difficult to maintain as a system grows: new files introduce responsibilities and dependencies, while architectural documentation may lag behind the implementation. Code-to-architecture (C2A) mapping addresses this problem by assigning source files to the modules of a reference architecture. The resulting mapping supports reflexion modeling, which compares implementation dependencies with architectural relations to identify divergence [39]. It therefore connects the modular decomposition used to reason about a system [49, 9] to the code in which that decomposition is realized.

Several sources of evidence can support a mapping. Identifiers express domain concepts and implementation roles. Contextual code representations capture information from declarations and operations. Static dependencies locate a file among other files with which it interacts, while directories record how developers organize the project [66, 6, 15]. These

sources need not agree: a utility may depend on several modules, terminology may be shared across responsibilities, and a directory may contain files assigned to different architectural modules.

The usefulness of an information source also depends on how it enters a predictor. A dependency embedding can encode graph structure before a classifier sees a file. Propagating over dependencies then supplies a second form of access to related files, with its own assumptions about direction, relation types, and aggregation. A gain from the embedding does not establish that this additional propagation will help. Similarly, agreement with a reference architecture may partly reflect agreement between that architecture and the directory structure, in which case aggregate scores do not show how a folder-aware model handles files whose directory is not indicative of their module. These distinctions motivate studying both the information supplied to a model and the changes caused by the way it processes that information.

Prior graph-based representation learning work on architecture recovery represents these sources in different ways, from Node2Vec random walks on the dependency graph [6] and graph attention with diffusion-based rewiring [24] to a graph autoencoder over identifier and folder tokens [23]. For mapping, prior work combines lexical and organizational features with dependencies in a single graph neural network [21, 22]. Because all signals enter that model together, and the predictors compared there differ in neighbor access as well as in architecture, it remains unclear how much each signal contributes and how it should be represented.

We examine this question through C2A mapping, formulated as a classification problem in which each file is assigned to one module of the reference architecture. We work in a transductive low-label setting, where a small sample of files covering every module is already mapped, and the model infers the remaining assignments. The model is trained on the labeled files alone. This setting allows us to examine how file representations and graph relations contribute under limited supervision.

Our evaluation uses ten Java, C, and C++ systems and covers lexical Word2Vec, structural Node2Vec, and contextual UniXcoder representations. These serve as inputs to an MLP, to a Heterogeneous Graph Transformer (HGT) [20], and, in the dependency comparison, to a neighbor MLP and a Graph Attention Network (GAT) [63]. The heterogeneous configurations contain file nodes and, where applicable, folder nodes and one aggregate member node per file. They vary both the context available to a file and where feature content is stored. The representations themselves differ in training, vector size, and source content, so their comparison is between representation pipelines and not between isolated information sources.

The study is guided by three research questions:

- **RQ1.** How does mapping performance vary across lexical, structural and contextual file representations?
- **RQ2.** How does adding member and folder nodes to file-level HGT change mapping performance, and how do folder-aware predictions compare with a directory rule?
- **RQ3.** How does using dependency neighbors change mapping performance, and how do the predictor and aggregation direction affect this change?

This study contributes an evaluation of C2A mapping that separates the information supplied to a model from the way the model uses it. Each research question varies one component while everything else, including the labeled splits and training settings, is held fixed: the file representation, the graph context available to HGT, and access to dependency neighbors within a fixed predictor. We also compare folder-aware models with a directory rule that uses the same labeled files and examine the files the rule assigns incorrectly, since aggregate scores alone do not show whether a gain exceeds what directory organization already provides. Code and data are available on GitHub[1].

2 Background

Software artifacts are naturally relational and multi-typed. A system contains folders, source files or classes, and finer-grained program elements such as methods and fields, connected through relations that include folder containment, file-to-member composition, and static code-level dependencies.

To preserve these distinctions, we represent each software system as a typed graph:

$$G = (V, E, \mathcal{A}, \mathcal{R}),$$

where V and E are the sets of artifact nodes and relations between them, respectively. The mappings $\tau : V \to \mathcal{A}$ and $\phi : E \to \mathcal{R}$ assign node and relation types. In our most detailed configuration, $\mathcal{A}$ contains folder, file, and aggregate member types, while $\mathcal{R}$ distinguishes folder containment, file-to-member composition, and a single dependency relation that merges all file-level static dependencies. The file-level graph is homogeneous, and adding folder or member nodes makes it heterogeneous.

A relation's meaning depends on its type and the types of its source and target nodes in G. We describe an edge $e_{s,t}$ from node s to node t by the meta-relation $\langle \tau(s), \phi(e_{s,t}), \tau(t) \rangle$. For example, $\langle \text{folder}, \text{contains}, \text{file} \rangle$ captures project organization, whereas $\langle \text{file}, \text{depends on}, \text{file} \rangle$ captures code-level coupling. These meta-relations preserve the distinction between structural organization and implementation dependencies, which a homogeneous graph with one node type and one relation type does not explicitly represent.

[1] `https://github.com/sse-lnu/hetmap`

HGT [20] exploits such meta-relations during message passing by conditioning attention and message transformation on the source node type, edge type, and target node type. GAT [63] also uses attention-based message passing, but its standard formulation does not explicitly distinguish node or relation types. HGT can also operate on a graph with a single node type and a single relation type, although this does not make its attention mechanism equivalent to that of GAT.

The initial node features determine what information is available before message passing, while graph structure determines which nodes exchange information. Depending on how they are encoded, a file's features can capture lexical evidence from its identifiers, structural evidence from its position in the dependency graph, or contextual evidence from its code.

A lexical embedding trained on identifier tokens captures naming patterns but does not explicitly encode dependency structure or implementation syntax and behavior. A graph embedding such as Node2Vec [16] instead encodes structural neighborhoods without using code content. Such embeddings have been used to capture dependency-based cohesion in architecture recovery [6, 24].

Contextual representations from pretrained code encoders such as UniXcoder [17] capture the syntactic and behavioral content of an implementation that lexical features miss, but their pretraining objectives do not target architectural grouping: files with similar code can belong to different modules. Contrastive learning adapts an encoder to a chosen notion of similarity by bringing designated positive pairs closer than other in-batch examples treated as negatives, using an InfoNCE objective [47]. Dependency-derived positive pairs encourage embeddings to reflect dependency proximity, which need not align with reference-module membership.

3 Methodology

This section presents the methodology for studying how architectural signals and graph design choices affect C2A mapping. We first formulate the mapping task and then describe the graphs and the structural, lexical, and contextual features used to initialize their nodes. We then present the MLP, HGT, and GAT predictors.

Each research question varies one of these components and holds the others fixed. RQ1 varies the file representation, using the MLP. RQ2 varies the graph variant for each representation, using HGT because it handles the node and relation types of the heterogeneous variants. RQ3 varies the predictor, comparing the neighbor MLP with GAT and HGT, and the dependency neighbors they receive.

3.1 Problem Formulation

We formulate the mapping task as node classification over a software graph $G = (V, E, \mathcal{A}, \mathcal{R})$, where $\mathcal{A}$ and $\mathcal{R}$ denote the sets of node and relation types. Let $V_f \subseteq V$ be the set of file nodes, which are the targets of the mapping task. A small subset $L \subset V_f$ is labeled with known architectural module assignments Y_L, and the remaining file nodes $U = V_f \setminus L$ are unlabeled. The task is to learn a model:

$$f_\theta : (G, L, Y_L) \rightarrow \hat{Y}_U, \tag{1}$$

where θ denotes the learnable parameters. The model is trained on the labeled files only, but all nodes of G, labeled and unlabeled, are part of the graph during training.

Each model computes a d-dimensional embedding z_i for every file $i \in V_f$, and the architectural module of a file is predicted from this embedding.

3.2 Graph Construction

We represent the software system as a graph of software artifacts connected by folder organization, member containment, and static dependencies.

To study the individual and combined contributions of folder and member information, we construct four graph variants that differ in the node types they contain. They are named after these types: F (file nodes only), FF (folder and file nodes), FM (file and member nodes), and FFM (folder, file, and member nodes). All variants retain the dependency relation. F is homogeneous, with one node type and one relation type, and the other three variants are heterogeneous. Figure 1 provides a visual overview.

3.2.1 Nodes and Relations

The complete graph (FFM) has three node types. File nodes represent source files and are the targets of C2A mapping. Because logical components are often implemented through namespaces or directory structures that group source code providing a particular functionality [54, 15], folder nodes represent directory levels of the project, with one node per level. Each file has one summarized member node covering the methods, fields, constructors, and variables of that file. This retains member-level information while avoiding the large and sparse graphs produced by representing individual members.

Four relations connect these nodes, as listed in Table 1. Three are structural and encode organization and containment, and the fourth encodes static dependencies. Folder nodes are connected through *parent_of* edges, forming chains such as `app` → `app/ui` → `app/ui/views`, and *contains* edges link each folder to its files. A *belongs_to* edge links each member node to its enclosing file.

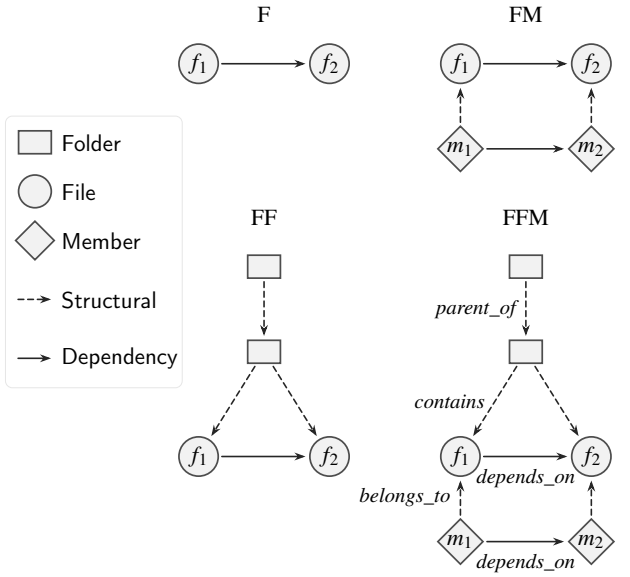


Fig. 1: Graph variants F, FM, FF, and FFM. Files f_i are the prediction targets, and each m_i aggregates the members of file f_i. Arrows point from source to target: dashed arrows are structural relations and solid arrows are dependencies. Relation names are shown on FFM only. The panels omit further dependency edges and folder levels; Table 1 lists the relations with the node types they connect. Feature construction also depends on the variant, as described in Section 3.3.

Messages along structural relations flow toward the file. Folder information passes down the hierarchy through *parent_of* and *contains* edges, and member information passes up to the file through *belongs_to* edges. No reverse edges are added, so folder and member nodes are not updated from files through structural relations.

The fourth relation, *depends_on*, encodes static dependencies, which provide direct evidence of how source files interact [6, 31]. Dependencies were extracted with the Depends tool [43], which analyzes the syntactic structure of the projects and identifies entities together with several categories of static relations. We merge all dependency types into a single relation, drop duplicate edges, and remove edges whose endpoints belong to the same file.

Dependency endpoints depend on the graph variant. In FM and FFM, dependencies connect either two files or two member nodes. Dependencies between a file and a member are resolved between their corresponding files. In F and FF, all member endpoints are replaced by their enclosing files, so all dependencies connect file nodes.

RQ1 uses no graph structure, and RQ3 uses the homogeneous file-level graph F. RQ2 uses all four variants to examine progressively richer graph structure, adding folder nodes in FF, member nodes in FM, and both in FFM.

Table 1: Graph relations and node types.

Relation	Source → target	Description
parent_of	folder → folder	Directory hierarchy.
contains	folder → file	Folder contains a file.
belongs_to	member → file	Links a member node to its file.
depends_on	file → file	File-level dependencies.
	member → member	Member-level dependency in FM and FFM.

3.3 Node Feature Initialization

Folder nodes are initialized in the same way in every setting. File and member nodes are initialized with one of the representations compared in RQ1: lexical embeddings from Word2Vec, structural embeddings from Node2Vec, or contextual embeddings from UniXcoder, used zero-shot or fine-tuned with dependencies.

Member information enters the model differently across representations. Word2Vec includes member identifiers in file vectors when member nodes are absent and places them in member nodes otherwise. UniXcoder provides member text only through member nodes, while Node2Vec assigns each member node its file's vector. Adding member nodes thus changes the placement or availability of information as well as the graph structure, so comparisons across graph variants are interpreted separately for each representation.

3.3.1 Folder Node Features

Folder nodes use a binary bag-of-words representation of the folder tokens along their paths, after removing common project-root tokens. For example, `app/ui/views` is represented by `app`, `ui`, and `views`. Including the tokens of its ancestors preserves each folder's hierarchical context.

3.3.2 Word2Vec (W2V): Lexical Features

Source code identifiers encode domain concepts and implementation roles, making them useful signals for C2A mapping [66]. Following Jabeen et al. [23], we extract tokens from file names and from the identifiers of members, namely methods, fields, constructors, and variables. We split camelCase, PascalCase, and snake_case identifiers, convert tokens to lowercase, and remove stopwords and non-alphabetic tokens. Each node vector is the mean of the skip-gram Word2Vec embeddings of its associated tokens [34].

In FM and FFM, file nodes use file-name tokens, while member nodes use member identifiers. In F and FF, member identifiers are included in the file vector instead, and the MLP uses this same file vector. Member identifiers therefore remain available in every variant.

3.3.3 Node2Vec (N2V): Structural Features

Chen et al. [6] observe that files in the same architectural module can occupy a dense local dependency region without being directly connected. They use random-walk embeddings to capture this longer-range structure for architecture recovery. In our earlier study of dependency-only recovery, which follows their setup, Node2Vec was among the best-performing learned representations [24]. Therefore, we use Node2Vec [16] embeddings computed with uniform random walks on the undirected file-level dependency graph as a structural initialization. These embeddings encode dependency structure without using identifiers or folder location.

In FM and FFM, each summarized member node receives the Node2Vec embedding of its corresponding file.

3.3.4 UniXcoder: Contextual Features

For contextual initialization, we use UniXcoder [17], a pre-trained code transformer. We parse each source file with Tree-sitter [5] using the corresponding Java, C, or C++ grammar. The syntax tree determines which source fragments are passed to the encoder, but is not itself an input to the model. Extraction produces two views. The file-level view contains type declarations, fields or global variables, and constructor signatures with short body excerpts where applicable. The member-level view contains methods or functions. Each view produces one vector per file. Member-level text is split into overlapping chunks whose embeddings are mean-pooled.

The zero-shot variant ($\mathrm{UXC_{ZS}}$) uses the pretrained weights. The dependency-guided variant ($\mathrm{UXC_{FT}}$) is fine-tuned separately for each system with a contrastive objective [47] using dependency-based positive pairs. Fine-tuning uses only the file-level views, and module labels do not enter the loss. The fine-tuned encoder then encodes both views and produces one vector per view for each file, so member-level text is encoded but is not part of the fine-tuning objective.

Positive pairs are drawn from the two-hop neighborhood to capture longer-range dependency structure [24], with dependencies treated as undirected. For each anchor file, two positives are sampled at random from the eight two-hop candidates that share the most one-hop neighbors with it. Files without a two-hop candidate fall back to a direct neighbor. Other files in the batch serve as negatives, except directly connected files, which are masked unless selected as the positive pair.

3.4 Mapping Models

We compare models that use file features alone with models that incorporate graph structure. RQ1 uses the MLP, RQ2 uses HGT, and RQ3 compares the neighbor MLP, GAT, and HGT. All models use the same classification head to predict architectural modules. Throughout, $\mathcal{N}(\cdot)$ denotes the neighborhood of a node.

3.4.1 Multilayer Perceptron (MLP)

The MLP predicts module logits directly from a file-level input vector. Each hidden layer computes:

$$h_i^{(k+1)} = \mathrm{ReLU}\big(W^{(k)} h_i^{(k)} + b^{(k)}\big), \tag{2}$$

with $h_i^{(0)} = x_i$ the file feature vector and $W^{(k)}, b^{(k)}$ the learnable parameters of layer k, and ReLU is the rectified linear unit [40]. The input x_i contains the single representation being evaluated.

For RQ3, we incorporate dependency neighborhood information in the MLP by concatenating each file's vector x_i with the mean $\bar{x}_{\mathcal{N}(i)}$ of its direct neighbors' vectors in the dependency graph, giving $[x_i \,\|\, \bar{x}_{\mathcal{N}(i)}]$. We refer to the MLP with this input as the neighbor MLP ($\mathcal{N}_{\mathrm{MLP}}$). With neighbors disabled, the input is x_i alone, as in RQ1, so enabling neighbors also doubles the input width of the first layer.

3.4.2 Heterogeneous Graph Transformer (HGT)

Node types have different feature spaces depending on the graph variant and initialization strategy. So HGT first maps the input vector x_t of each node t into a shared hidden space using a type-specific linear transformation followed by ReLU, which gives its initial representation $h_t^{(0)}$. One HGT layer [20] then updates node representations through multi-head attention over their neighbors.

For a target node t and a source node s connected by an edge e, the layer computes a message $M(s,e,t)$ and an attention weight $\alpha(s,e,t)$, and aggregates them as:

$$m_t = \sum_{s \in \mathcal{N}(t)} \alpha(s,e,t)\, M(s,e,t), \tag{3}$$

$$M(s,e,t) = W^{\mathrm{MSG}}_{\phi(e)} V_{\tau(s)}\, h_s,$$

$$\alpha(s,e,t) = \underset{s \in \mathcal{N}(t)}{\mathrm{softmax}} \frac{\big(K_{\tau(s)} h_s\big)^{\top} W^{\mathrm{ATT}}_{\phi(e)} \big(Q_{\tau(t)} h_t\big)\, \mu_{\langle \tau(s), \phi(e), \tau(t) \rangle}}{\sqrt{d_h}}.$$

where $\tau(\cdot)$ is the node type, $\phi(e)$ is the edge type, V, K, and Q are type-specific linear transformations, W^{MSG} and W^{ATT} are edge-type-specific matrices, μ is a learned prior for each meta-relation, d_h is the head dimension, and attention-head indices are omitted for brevity. These parameters allow a *contains* edge from a folder and a *depends_on* edge from a dependent file to receive different attention weights.

After combining the attention-head outputs, we denote the resulting message by m_t. In our implementation, this message is combined with the node's projected input $h_t^{(0)}$ through a residual connection [19] followed by layer normalization:

$$h_t^{(1)} = \mathrm{LayerNorm}\left(m_t + h_t^{(0)}\right). \tag{4}$$

Folder and member nodes, when present, contribute to file representations through message passing.

3.4.3 Graph Attention Network (GAT)

GAT [63] aggregates neighbor representations using attention parameters shared across neighbors within each head, without the node-type- or edge-type-specific parameters of HGT. The outputs of the attention heads are concatenated. Our GAT implementation uses the same input projection, residual connection, and layer normalization as HGT, while replacing its neighbor aggregation mechanism. We add no self-loops; the residual connection retains the node's own representation.

3.4.4 Classification Head

All three predictors use the same classification head. The file embedding $z_i \in \mathbb{R}^d$, obtained from the last hidden layer of the MLP or the message-passing layer of GAT and HGT, is passed to a linear layer that produces one score per architectural module. A softmax converts these scores into a probability distribution:

$$\begin{aligned} p_i &= \mathrm{softmax}\big(W_c z_i + b_c\big), \\ \hat{y}_i &= \arg\max_m p_{i,m}, \end{aligned} \tag{5}$$

where $W_c \in \mathbb{R}^{|M|\times d}$, $b_c \in \mathbb{R}^{|M|}$, and $|M|$ is the number of architectural modules. The predicted module $\hat{y}_i$ is the one with the highest probability. Training minimizes categorical cross-entropy over the labeled file nodes.

4 Experiments

This section describes the benchmark systems, the experimental configuration, the directory-based reference predictor, and the evaluation procedure used to answer the three research questions.

4.1 Benchmark Systems

We evaluate the models on ten open-source systems with published ground-truth (GT) mappings, which have been used in prior work on architecture recovery and C2A mapping [23, 46]. They differ in programming language, domain, size, and architectural granularity. We use published mappings because reliable C2A mappings require substantial manual effort and expert validation [15], and because they allow comparison with previous studies.

Table 2 summarizes the system and graph properties. File-level dependency homophily $\mathcal{H}$ is the proportion of unique directed file pairs connected by a static dependency whose source and target files fall within the same GT module, from 0 (every dependency crosses a module boundary) to 1 (every dependency stays within a module). The mean degree $\langle k \rangle = 2|E|/|\text{Files}|$ counts each of the $|E|$ directed file-level dependency edges at both of its files.

Table 2: Subject systems and their file-level dependency graph properties. M: number of modules in the GT; $|E|$: number of directed file-level static dependencies; $\mathcal{H}$: dependency homophily; $\langle k \rangle$: mean degree.

	System	Version	M	Files	$\lvert E\rvert$	$\mathcal{H}$	$\langle k \rangle$
Java	A.UML	v0.24	14	766	3 754	0.55	9.80
	C.Img	v1.0a2	21	329	1 565	0.61	9.51
	JabRef	v3.7	6	1 013	6 126	0.56	12.09
	Lucene	r1075001	7	1 054	6 424	0.68	12.19
	SH-3D	r002382	9	167	1 220	0.36	14.61
	T.Mates	v5.11	15	778	6 721	0.34	17.28
C	Bash	4.2	14	292	1 010	0.58	6.92
	HDF	0e196f	9	153	535	0.35	6.99
C++	HDC	46ff87	11	207	543	0.63	5.25
	Chrome	svn-171054	69	18 343	145 492	0.58	15.86

The Java systems come from the benchmark of Olsson et al. [46]. JabRef and TeamMates (T.Mates) were studied at the SAEroCon workshop in 2016 and 2017, where system experts provided the architectures and mappings[2]. Lucene was studied by Lenhard et al. [30], and the Commons Imaging (C.Img) mapping was contributed as a test case for s4rdm3x [45] [3].

For Bash, we use the mapping curated by Garcia et al. [15], who recovered an initial decomposition from documentation and source code and refined it with system experts through a certification process. The same work provides the Chromium mapping, developed with developers over approximately two years [31].

The HDF and HDC mappings were developed with industry partners and are part of the SARIF benchmark of Zhang et al. [66], available in its replication package[4].

4.2 Experimental Configurations

We repeat each experiment 100 times, each with a different initial labeled set. In each run, we randomly sample 5% of the file nodes, with a minimum of 30 and a maximum of 200 files, and add one file from each GT module not represented in the sample. We use the same 100 initial labeled sets across all experiments and configurations, which allows paired comparisons. This procedure imposes a bounded labeling budget rather than a constant labeled proportion: before the module-coverage additions, the random sample covers approximately 9–20% of the files in the five systems subject to the floor (C.Img, SH-3D, Bash, HDF, and HDC) and approximately 1% in Chromium, the only system subject to the cap.

[2] https://github.com/sebastianherold/SAEroConRepo
[3] Java labels are available at: https://github.com/h0bb3/s4rdm3x
[4] https://github.com/anonymous2f4a9d/SARIF_FSE23

Input representations. For each system, Word2Vec embeddings are 128-dimensional vectors trained for 20 epochs with a context window of five on the file names and identifiers.

Node2Vec learns 128-dimensional vectors from uniform random walks ($p = q = 1$) on the undirected file-level dependency graph, with walk length 30, 200 walks per file, window 15, five skip-gram epochs, and five negative samples. Chromium uses 100 walks per file and one negative sample.

For UXC_{FT}, UniXcoder is fine-tuned separately for each system with an InfoNCE temperature of 0.05, AdamW with a learning rate of 2×10^{-5}, a batch size of eight, and two epochs with a fixed seed. Only the upper four of the twelve encoder layers are updated, while the token embeddings and the lower eight layers stay frozen. Both UniXcoder variants encode file inputs with a 512-token limit and member inputs in overlapping 256-token windows with a stride of 128, mean-pooled before normalization, producing 768-dimensional features.

All input representations are generated once per system and reused in every run and experiment.

Training models. All models are trained with Adam at a learning rate of 0.001 for 200 epochs. The MLP has two hidden layers of 256 dimensions, and GAT and HGT use one layer with 256 hidden dimensions and four attention heads.

Controlled comparisons. RQ1 trains the MLP on each of the four representations, which isolates the information in the file representation because the MLP uses no graph structure.

RQ2 trains HGT on all graph variants (Figure 1) with Word2Vec, Node2Vec, and UXC_{FT} features, using directed dependency edges. We additionally train all four HGT variants with two layers to test whether one layer limits what a file receives from its neighbors. We also train **FD**, an HGT model on graph F whose UXC_{FT} file vector is extended with a binary folder-path vector, including ancestor folders. This vector uses the same tokenizer and vocabulary as the folder-node features (Section 3.3), with one dimension per distinct folder token in the system. FD provides folder information without folder nodes.

RQ3 consists of controlled comparisons on the file graph, with all four representations. Within each predictor (MLP, GAT, and HGT), we vary only whether dependency neighbors are available and in which direction information flows. We keep the file representations, initial labeled sets, and training settings fixed and compare three settings. With neighbors *disabled*, each file uses only its own representation. With *directed* aggregation, a dependency edge from file u to file v, where u depends on v, carries information from u to v. Each file therefore receives information from the files that depend on it. With *undirected* aggregation, we also add the reverse edge, allowing each file to receive information from files it depends on. For the MLP, these settings determine which files contribute to the neighbor vector; for GAT and HGT, they determine which files send messages.

The MLP with neighbors disabled is the RQ1 classifier, and HGT with directed neighbors on the file graph is the F configuration of RQ2, so these runs are shared. In total, the experiments comprise 58,000 runs.

4.2.1 Directory Hierarchy Baseline

To assess how much folder-based configurations exploit files in the same directory sharing a module, we use a rule-based predictor based on the directory hierarchy and the initial labeled files. For each unlabeled file, it assigns the most common module among labeled files in the same directory. If none are present, it searches the directory subtree and then the subtrees of its parent directories until it finds labeled files. Ties are resolved by the module's frequency in the complete initial labeled set and then alphabetically. If no labeled files are found, the predictor assigns the most common module in the complete initial labeled set. It uses the directory hierarchy, but not directory-name text, source code, embeddings, dependencies, or test labels.

We apply the predictor to all 100 initial labeled sets and compare it with HGT using UXC_{FT} features on the directed FM and FFM graphs and with FD. For each system, we report the accuracy of the predictor, FM, FFM, and FD, which equals micro-F1 because each file has exactly one module label. For the files that the predictor assigns incorrectly, we also report the accuracy of each compared model on those files.

4.3 Performance Evaluation

The benchmark systems have imbalanced module sizes, so overall accuracy can hide poor performance on smaller modules. We therefore report both macro-F1 and micro-F1. F1 is the harmonic mean of precision and recall, which measure the correctness of predicted assignments and the coverage of true assignments, respectively. Macro-F1 averages the F1 scores of individual modules, giving each module equal weight regardless of size. Micro-F1 aggregates predictions across modules, giving each file equal weight. Reporting both captures performance across modules as well as overall file-level performance.

Because compared configurations use the same initial labeled sets, we pair runs by split. We use the two-sided Wilcoxon signed-rank test [64], a non-parametric test, to compare configurations separately for each system and metric without assuming normally distributed score differences. We apply Holm's correction across the ten systems for each metric and pair of compared configurations, using a significance level of $\alpha = 0.05$.

To quantify the magnitude of the differences, we also compute Cliff's δ [10] from the two sets of 100 scores. We

classify its magnitude as negligible (N, $|\delta| < 0.147$), small (S, $0.147 \leq |\delta| < 0.33$), medium (M, $0.33 \leq |\delta| < 0.474$), or large (L, $|\delta| \geq 0.474$) [56].

5 Results

This section reports the results for code-to-architecture mapping across ten subject systems. RQ1 compares lexical, structural, and contextual file representations using an MLP. RQ2 examines how mapping performance changes when Word2Vec, Node2Vec, and UXC_{FT} initialize HGT with file, member, and folder context, and analyzes errors relative to directory-based predictions. RQ3 uses controlled experiments to examine how using dependency neighbors changes mapping performance, and how the predictor and aggregation direction affect this change.

For RQ1 and RQ2, we report median macro- and micro-F1 scores over the 100 runs per system and configuration, and differences between these medians in F1 percentage points, with Cliff's δ alongside the significance test. For RQ3, we report the median of the paired differences between conditions over the 100 splits. Summary tables give the median of these per-system values across the ten systems. F1 is computed only on the held-out test files of each split.

5.1 Comparing Lexical, Structural, and Contextual File Representations (RQ1)

RQ1 examines how much predictive information each file representation provides before additional graph-based message passing. Table 3 reports the median macro-F1 and micro-F1 scores obtained with Word2Vec, Node2Vec, and UniXcoder variants: UXC_{ZS} and UXC_{FT}. Each representation is evaluated using the MLP.

Table 3: Median macro-F1/micro-F1 scores (%) across 100 runs for the MLP using W2V, UXC_{ZS}, UXC_{FT}, and N2V. Highlighted values indicate the highest score for each system and metric.

System	W2V	UXC_{ZS}	UXC_{FT}	N2V
A.UML	34 / 57	36 / 60	46 / 69	54 / 72
C.Img	23 / 39	33 / 52	42 / 62	71 / 80
JabRef	44 / 78	39 / 76	43 / 79	40 / 74
Lucene	57 / 71	54 / 66	63 / 76	66 / 79
SH-3D	22 / 41	48 / 64	47 / 67	43 / 58
T.Mates	30 / 55	37 / 68	51 / 79	41 / 68
Bash	7 / 29	23 / 42	27 / 49	42 / 61
HDF	8 / 22	42 / 44	46 / 48	53 / 57
Chrome	14 / 48	14 / 48	31 / 66	41 / 71
HDC	33 / 53	30 / 51	46 / 70	43 / 69

Macro-F1 is lower than micro-F1 for every system and representation. Node2Vec achieves the highest median macro- and micro-F1 scores on six systems. On the other four systems, UXC_{FT} obtains the highest median micro-F1, while the highest median macro-F1 is held by UXC_{FT} on T.Mates and HDC, Word2Vec on JabRef, and UXC_{ZS} on SH-3D.

Statistical significance of observed differences. Table 4 summarizes the pairwise differences across all ten systems, with the number of systems favoring each method and the number of Holm-significant differences in each direction.

Table 4: Summary of pairwise differences in F1 points across ten systems. Δ is the median difference (first method minus second). +ve is the number of systems favoring the first method, followed in parentheses by the numbers of significant positive (↑) and negative (↓) differences.

	Macro F1		Micro F1	
Comparison	Δ	+ve (↑/↓)	Δ	+ve (↑/↓)
$\text{UXC}_{\text{ZS}}-\text{W2V}$	+5.2	6 (6/4)	+7.5	6 (6/3)
$\text{UXC}_{\text{FT}}-\text{UXC}_{\text{ZS}}$	+9.1	9 (9/0)	+9.2	10 (10/0)
$\text{UXC}_{\text{FT}}-\text{W2V}$	+17.5	9 (9/0)	+19.2	10 (9/0)
$\text{UXC}_{\text{FT}}-\text{N2V}$	−4.9	4 (4/6)	−3.3	3 (3/6)
N2V−W2V	+20.3	9 (9/1)	+16.2	9 (9/1)

We first compare Word2Vec and UXC_{ZS}, the two representations that do not encode dependencies. They do not separate consistently: the significant differences fall in both directions, although UXC_{ZS} is favored on more systems. On macro-F1, UXC_{ZS} is significantly stronger on A.UML (medium effect), C.Img, SH-3D, T.Mates, Bash, and HDF (large effects), whereas Word2Vec is significantly stronger on JabRef (large effect), Lucene and Chrome (medium effects), and HDC (small effect). On micro-F1, UXC_{ZS} is significantly stronger on the same six systems, and Word2Vec is significantly stronger on JabRef and Lucene (large effects) and HDC (small effect); the difference in Chrome is not significant.

Fine-tuning significantly improves UniXcoder over UXC_{ZS} on micro-F1 for all ten systems and on macro-F1 for nine, predominantly with large effects. The effects are medium on HDF (both metrics) and on SH-3D micro-F1; the macro-F1 difference on SH-3D is not significant.

Both representations encoding dependency information are generally stronger than the lexical representation. Node2Vec significantly outperforms Word2Vec on both metrics for nine systems, with JabRef as the exception, where Word2Vec is significantly stronger. Similarly, UXC_{FT} significantly outperforms Word2Vec on both metrics for nine systems; on JabRef, the difference is not significant on either metric.

Node2Vec is significantly stronger than UXC_{FT} on both metrics for A.UML, C.Img, Lucene, Bash, HDF, and Chrome, whereas UXC_{FT} is significantly stronger on JabRef, SH-3D, and T.Mates; most of these differences have large effects. On

HDC, UXC_{FT} is significantly stronger on macro-F1 with a small effect, and the micro-F1 difference is not significant.

Answer to RQ1.

Representations that encode dependencies, either directly through random walks (Node2Vec) or through dependency-guided fine-tuning of UniXcoder, provide the strongest mapping performance overall. Fine-tuning consistently improves UniXcoder over its zero-shot representation and produces the best macro-F1 on two systems. Word2Vec and UXC_{ZS}, which do not encode dependencies, differ from each other in both directions, and Word2Vec is weaker than Node2Vec and UXC_{FT} on most systems; Word2Vec nevertheless remains stronger than Node2Vec on JabRef. Thus, no single file representation performs best across all systems and metrics.

5.2 Message Passing over Dependency Edges and through Member and Folder Nodes (RQ2)

RQ2 examines how mapping performance changes for each representation when message passing over the file dependency graph is added and the graph is extended with folder and member nodes. Figure 2 reports median macro- and micro-F1 for the MLP and HGT under F, FF, FM, and FFM. We use Word2Vec, Node2Vec, and the fine-tuned variant UXC_{FT}, which improves over UXC_{ZS} on nearly all systems in RQ1.

Table 5 summarizes five steps: MLP to F, adding members (F to FM, FF to FFM), and adding folders (F to FF, FM to FFM). For each system, Δ is the difference between the two configuration medians over 100 runs. The table reports the median of these ten system-level differences, the number of systems with a positive difference, and the number of systems with a significant gain or loss.

File-level HGT versus the MLP. Moving from the MLP to file-level HGT improves Word2Vec and UXC_{FT} more than Node2Vec. For Word2Vec and UXC_{FT}, the improvement is positive on nine systems on both metrics, and the only significant loss is UXC_{FT} on HDC macro-F1. For Node2Vec, the changes are small, and significant losses occur on Lucene and HDC for both metrics and on HDF for macro-F1, but significant gains still outnumber losses on both metrics.

Member-level context. Adding member nodes to F lowers median macro-F1 and micro-F1 for every representation, most strongly for Word2Vec, with large effects on all ten systems. Significant macro-F1 losses outnumber gains for every representation. Adding member nodes also changes how features enter the model, depending on the representation (Section 3.3); for example, the UXC_{FT} member vectors come from an encoder fine-tuned on file-level text. In addition, with one HGT layer, member-level dependencies cannot affect file predictions, so we test two layers below. These results therefore concern the evaluated member configuration, not member information in general.

Folder-level context. Adding folder nodes produces the largest and most consistent changes. FF improves over F, and FFM improves over FM, on both metrics for every system and representation. All 120 system–representation–metric comparisons are significant with a large effect, and each improves in at least 96 of the 100 paired runs.

Within the folder-containing configurations, the effect of adding member nodes depends on the representation. FFM improves over FF for Node2Vec and UXC_{FT} on both metrics, with small median gains, but lowers both metrics for Word2Vec. The significant macro-F1 losses for Node2Vec and UXC_{FT} occur on JabRef, Bash, and Chrome. For micro-F1, Node2Vec loses significantly only on Bash, and UXC_{FT} on no system.

5.2.1 Depth of message-passing layers

To test whether using one HGT layer limits the results, we repeated F, FF, FM, and FFM with two layers, using the same labeled sets, representations, and training settings (Table 6). For Word2Vec, the largest median macro-F1 improvements occur in FM and FFM. For UXC_{FT}, FF improves more than FFM, so the improvement is not specific to member nodes. Node2Vec shows little change in either member-containing configuration, but F is significantly lower on macro-F1 on five systems.

At matched two-layer depth, adding members to F still lowers median macro-F1 for all three representations. Adding members to FF remains negative for Word2Vec, while the small median gain for UXC_{FT} shrinks and that for Node2Vec becomes slightly negative. Folder nodes retain the largest gains: both folder additions improve both metrics on all ten systems for every representation, with significant large effects. Thus, additional depth improves some configurations but does not establish a consistent benefit from adding member nodes.

5.2.2 Organizational information and the directory baseline

The large gains from adding folder nodes show that directory organization is informative for the evaluated mappings. We first test whether these gains require folder nodes using FD, which adds folder-path features directly to the UXC_{FT} file vectors in graph F. We then examine whether the gains also help files misclassified by directory information alone, comparing FM, FFM, and FD against the directory hierarchy baseline (Section 4.2.1). FM and FFM differ only in the inclusion of folder nodes. Table 7 reports overall accuracy for the baseline and the three models, and model accuracy on files misclassified by the baseline. FD improves over F on every system with a large effect. Compared with FD, FF has lower

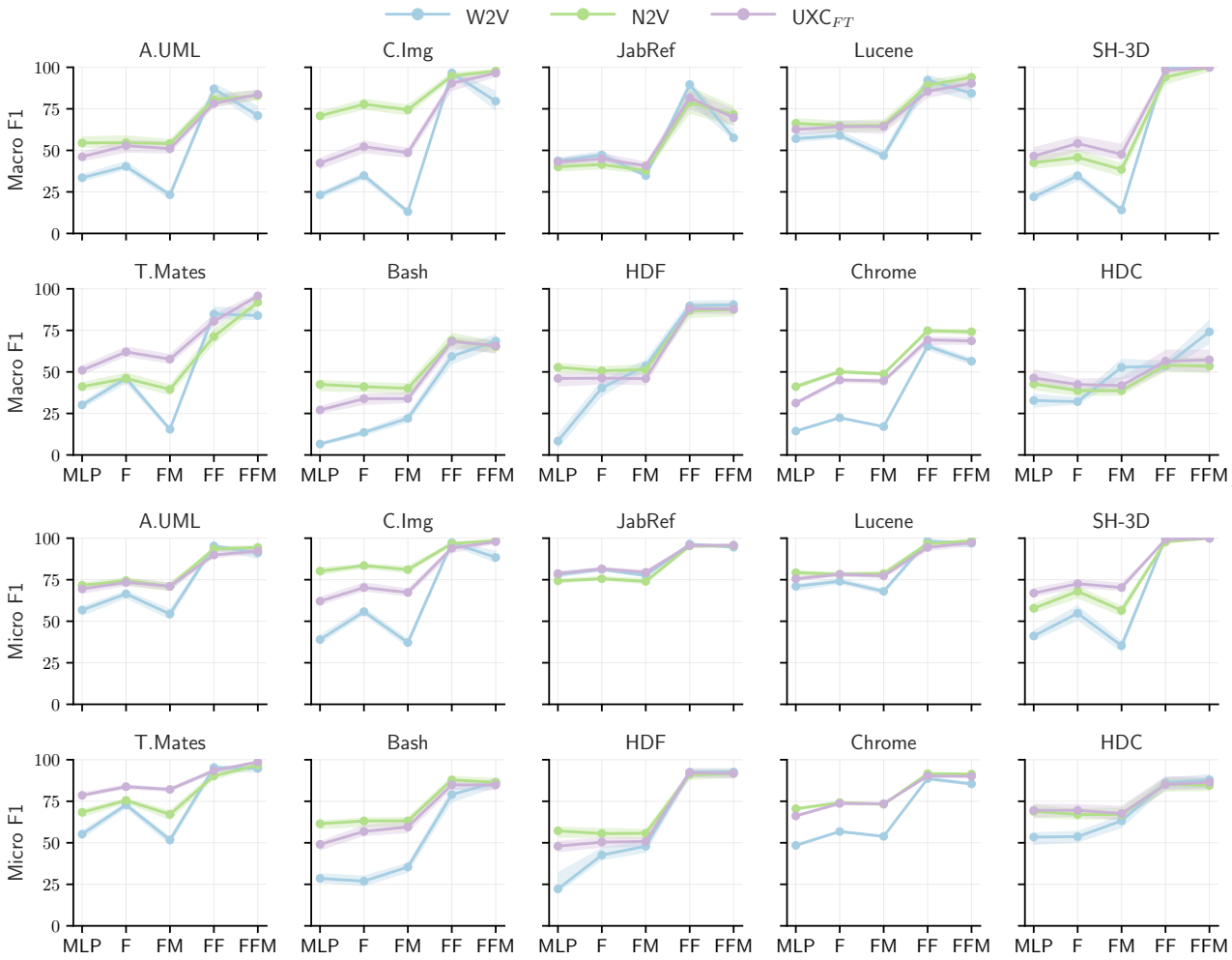


Fig. 2: Median macro-F1 (top) and micro-F1 (bottom) for each representation, per system, across the MLP and HGT with increasing graph structure (F, FF, FM, FFM). Shaded areas denote interquartile ranges across the 100 runs.

Table 5: Change in F1 as graph structure is added, by representation. Each cell gives the median over ten systems of the difference between configuration medians in F1 points, followed in parentheses by the number of systems with a positive difference and the number of significant positive/negative differences. Blue shading marks gains and purple losses, darker for larger changes.

	W2V		N2V		UXC$_{FT}$	
Step	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
F − MLP	+7.5 (9; 9/0)	+9.0 (9; 8/0)	+0.8 (6; 5/3)	+2.2 (7; 7/2)	+6.7 (9; 8/1)	+4.6 (9; 9/0)
FM − F	−12.2 (3; 3/7)	−4.9 (3; 3/7)	−1.2 (2; 0/5)	−1.3 (3; 0/6)	−1.2 (2; 1/6)	−1.8 (2; 1/7)
FFM − FF	−4.4 (3; 2/5)	−1.0 (3; 2/5)	+1.6 (6; 5/3)	+0.8 (7; 5/1)	+1.3 (6; 5/3)	+1.1 (7; 5/0)
FF − F	+44.4 (10; 10/0)	+32.1 (10; 10/0)	+25.6 (10; 10/0)	+18.7 (10; 10/0)	+29.9 (10; 10/0)	+16.5 (10; 10/0)
FFM − FM	+43.0 (10; 10/0)	+39.9 (10; 10/0)	+28.9 (10; 10/0)	+22.3 (10; 10/0)	+32.3 (10; 10/0)	+20.6 (10; 10/0)

median macro-F1 on eight systems, significantly so on six, and no significant gains. Folder nodes and relations therefore do not improve on supplying folder-path information directly as file features in this configuration.

The baseline reaches 89 to 100% accuracy on every system. Without folder nodes, FM is 14 to 43 points below it. FFM and FD are within 7 points of it on nine systems and exceed it only on Lucene. Folder information therefore brings both FFM and FD close to the accuracy of the directory hierarchy baseline on most systems.

Directory membership closely matches module membership: on eight systems, 99 to 100% of files belong to the

Table 6: Effect of a second message-passing layer on HGT. Δ is the median over ten systems of the change in median F1 (points) from one to two layers. +ve is the number of systems with a positive change, followed by the numbers with a significant gain (↑) and loss (↓). Shading marks comparisons with at least five significant gains and at least twice as many significant gains as losses.

		Macro-F1		Micro-F1	
Input	Graph	Δ	+ve (↑/↓)	Δ	+ve (↑/↓)
W2V	F	+0.5	6 (3/4)	0.0	6 (4/3)
	FF	+2.4	8 (6/1)	+0.5	6 (6/0)
	FM	+3.5	6 (6/3)	+2.2	8 (7/2)
	FFM	+10.3	6 (6/3)	+1.1	6 (6/3)
N2V	F	−0.6	4 (2/5)	−0.1	4 (2/4)
	FF	+1.8	7 (5/2)	+0.4	6 (4/3)
	FM	0.0	5 (3/3)	+0.1	5 (3/3)
	FFM	+0.3	5 (4/3)	−0.2	3 (2/4)
UXC_{FT}	F	0.0	6 (2/1)	+0.8	8 (7/1)
	FF	+3.4	8 (8/2)	+1.4	10 (7/0)
	FM	−0.2	5 (1/2)	0.0	5 (3/3)
	FFM	+2.6	7 (7/1)	+1.0	8 (8/1)

Table 7: Accuracy of the directory baseline (Dir.) and of HGT with UXC_{FT} features on FM, FFM, and FD, median over 100 runs. *n* is the median number of test files per split that the baseline misclassifies; the columns under it give model accuracy on those files, over splits with at least one baseline error. Purity is the share of files that belong to the majority module of their directory.

	Accuracy (%)				Baseline misclassifications				
System	Dir.	FM	FFM	FD	*n*	FM	FFM	FD	Purity
A.UML	99	71	92	93	11	9.1	21.7	22.5	100.0
C.Img	100	67	98	99	0	–	–	–	100.0
JabRef	97	79	96	93	27	45.3	26.3	25.0	99.1
Lucene	92	77	97	95	86	52.9	86.9	81.7	100.0
SH-3D	100	70	100	99	0	–	–	–	100.0
T.Mates	100	82	99	99	0	–	–	–	100.0
Bash	92	60	85	86	21	24.6	26.1	27.8	91.8
HDF	93	51	92	93	8	27.3	50.0	40.0	100.0
Chrome	97	74	90	94	574	19.8	20.0	23.3	99.9
HDC	89	68	87	87	19	20.0	12.5	10.8	96.1

majority module of their directory, and only Bash and HDC have lower purity. Files that depart from the majority module of their directory are therefore rare on most systems, which limits what the analysis can show about them.

On the files the baseline gets wrong, FFM gets most of them wrong on A.UML, JabRef, Bash, Chrome, and HDC, about half on HDF, and most of them right only on Lucene. Compared with FM, FFM is more accurate on these files on A.UML, HDF, and Lucene, similar on Bash and Chrome, and less accurate on JabRef and HDC. JabRef illustrates the distinction between overall accuracy and accuracy on baseline errors: FFM improves over FM by 16 percentage points overall but falls by 19 points on these files. FD is within 10 points of FFM on these files on every system. This analysis does not isolate files whose module differs from the majority module of their directory, so it does not show how well FFM handles such exceptions.

Answer to RQ2.

The effect on mapping performance depends on what is added and on the representation. Moving from the MLP to file-level HGT improves Word2Vec and UXC_{FT} more than Node2Vec. Member nodes give no consistent gain and often lower performance, most strongly for Word2Vec. Folder nodes provide the largest gains, with significant large effects on all ten systems for all three representations. The findings for member and folder nodes remain with two HGT layers.
The directory hierarchy baseline achieves accuracy similar to FFM on most systems, and folder nodes do not consistently outperform folder information supplied directly as file features (FD). On most systems with directory-baseline errors, FFM misclassifies most of these files, and on some it is less accurate than FM. Strong folder-based scores therefore do not establish reliable performance on directory-baseline errors, and the analysis does not isolate files whose directory location is insufficient or misleading.

5.3 Dependency Neighbors, Predictors, and Aggregation Direction (RQ3)

RQ2 shows that the effect of graph context depends on the representation, configuration, and system. However, its comparison of the MLP with file-level HGT changes both the predictor and the availability of dependency messages. RQ3 therefore compares each predictor with dependency neighbors disabled and enabled, using HGT, GAT, and a neighbor MLP ($\mathcal{N}_{MLP}$). It also compares directed and undirected aggregation to examine whether the benefit depends on how dependency neighbors are aggregated. We evaluate all three predictors with all four input representations.

Differences are paired changes in F1 points, enabled minus disabled; positive values favor neighbor information. For direction, differences are undirected minus directed; positive values favor undirected neighbors. We report the median paired difference over the 100 splits. With neighbors disabled, the neighbor MLP reduces to the MLP of RQ1, and GAT and HGT receive no dependency messages.

How does the benefit of dependency neighbors vary across predictors and representations? Table 8 shows the change in F1 when neighbor information is enabled, for each predictor and input representation. With undirected neighbors, the median change is positive in every cell on both metrics. GAT and the $\mathcal{N}_{MLP}$ improve on at least eight systems in every cell and are never significantly lower, whereas HGT gains

least with every representation and is significantly lower on a few systems with Node2Vec and Word2Vec. With directed neighbors, the median macro-F1 is lower with Node2Vec for all three predictors, by 0.6 to 2.8 points, and significant losses occur on up to six systems. For GAT and the $\mathcal{N}_{\text{MLP}}$, the gain from undirected neighbors is smallest with Node2Vec and largest with UXC_{ZS}.

This comparison differs from RQ2, where file-level message passing (F) is compared with the MLP, which changes the predictor together with the available structure. Here the predictor is fixed, and dependency neighbors are enabled and disabled, so a gain of F over the MLP does not imply that HGT benefits from dependency neighbors. With Node2Vec, for example, F raises the median macro-F1 over the MLP, whereas HGT with directed neighbors is lower than HGT without neighbors.

Does the direction of neighbor aggregation matter? The preferred direction depends on the system. Undirected aggregation gives the higher median in almost every cell of Table 8. However, several systems favor directed aggregation. Figure 3 shows gains from undirected aggregation on five to eight systems per combination of predictor and input representation, with significant differences in both directions.

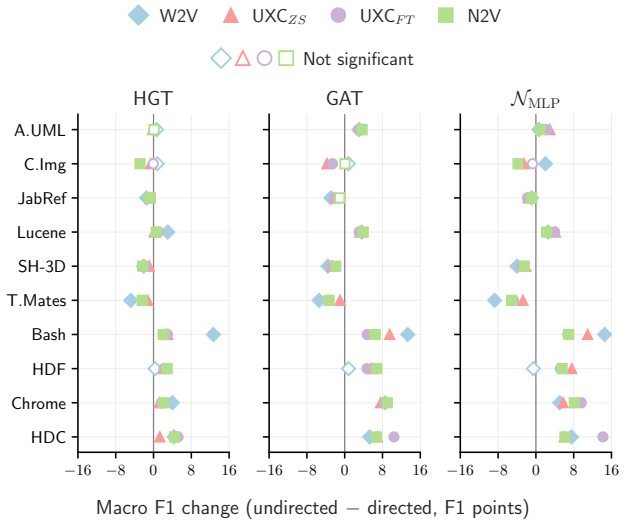


Fig. 3: Macro-F1 differences (undirected − directed), in F1 points, by predictor and input representation. Each point shows median paired differences over 100 splits; negative values favor directed aggregation. Hollow markers indicate non-significant differences.

On macro-F1, SH-3D, T.Mates, and JabRef score higher with directed aggregation in all 12 combinations, and C.Img in eight. Lucene, Bash, Chrome, and HDC score higher with undirected aggregation in all 12 combinations, and A.UML and HDF in 11. The largest differences are 14.6 points in favor of undirected aggregation (Bash, Word2Vec, $\mathcal{N}_{\text{MLP}}$) and 8.7 points in favor of directed aggregation (T.Mates, Word2Vec, $\mathcal{N}_{\text{MLP}}$). Micro-F1 shows the same directional pattern for seven systems, while JabRef, C.Img, and A.UML show mixed results. Adding reverse edges also expands the neighborhood, so this comparison does not isolate propagation direction from neighborhood size.

Which predictor scores highest? Figure 4 pools the ten systems with undirected neighbors: each point is the median of one system and the boxes show the spread across systems. GAT has the highest median macro-F1 with every representation and the highest median micro-F1 with every representation except Node2Vec, where HGT is marginally higher. GAT also scores highest most often: in each split we count which of the four predictors scores highest, and GAT has the largest share of wins, averaged over systems, with every representation and on both metrics. All three predictors with neighborhood aggregation exceed the MLP without neighbors in every median comparison. As without neighbors, UXC_{FT} has the highest median macro-F1 across systems for every predictor, with GAT and UXC_{FT} giving the highest scores overall.

Most of the variation in F1 lies between systems and not between runs of one system. The choice of predictor matters most for Word2Vec and UXC_{ZS}, the two representations without dependency information, and less for Node2Vec and UXC_{FT}.

Answer to RQ3.

Dependency neighbors do not benefit all predictors equally. With undirected aggregation, median changes are positive for every predictor and representation, but GAT and the $\mathcal{N}_{\text{MLP}}$ improve more consistently across systems than HGT, which shows significant losses in some cases. Directed aggregation is less reliable, particularly with Node2Vec, where median macro-F1 decreases for all three predictors. Although undirected aggregation generally gives larger gains, it is not preferable on every system: several systems consistently favor directed aggregation across predictors and representations. These comparisons also change neighborhood size, so they do not isolate direction alone. With undirected neighbors, GAT has the highest median macro-F1 across systems for every representation, and UXC_{FT} remains the strongest representation by this measure for every predictor.

6 Discussion

The research questions examine where architectural signal resides and how a mapping model can use it. RQ1 compares file representations before message passing, RQ2 examines file, member, and folder context and compares the folder gains with a directory-based rule, and RQ3 examines how

Table 8: Change in F1 when neighbor information is enabled, by input vector and graph condition. Each cell gives the median over ten systems of the median paired change in F1 points over runs, followed in parentheses by the number of systems with a positive median change and the numbers of significant positive and negative changes. D: directed (source-to-target); UD: undirected; off: neighbor information disabled. Shading marks the median change: blue for gains, purple for losses, and darker for larger changes.

		N2V		W2V		UXC_{ZS}		UXC_{FT}	
Model	Contrast	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
HGT	D−off	−0.6 (4; 4/6)	+1.6 (6; 6/4)	+0.2 (5; 4/5)	+1.8 (7; 7/3)	+1.6 (9; 7/0)	+2.7 (10; 10/0)	+0.7 (7; 6/3)	+1.8 (8; 8/0)
	UD−off	+0.5 (8; 5/2)	+0.8 (9; 8/0)	+2.9 (7; 7/3)	+4.5 (8; 8/2)	+1.8 (10; 10/0)	+2.5 (10; 10/0)	+0.9 (10; 8/0)	+1.6 (10; 10/0)
	UD−D	+0.3 (6; 5/4)	+0.1 (5; 3/4)	+0.7 (7; 4/3)	+1.5 (6; 5/4)	0.0 (5; 4/3)	+0.1 (5; 5/4)	+0.6 (6; 5/3)	+0.2 (5; 4/4)
GAT	D−off	−2.8 (4; 3/6)	−0.8 (3; 2/5)	+1.5 (8; 6/2)	+2.4 (7; 7/1)	+3.8 (10; 9/0)	+6.2 (10; 10/0)	+0.7 (6; 4/2)	+2.2 (8; 7/1)
	UD−off	+1.4 (9; 8/0)	+1.4 (10; 10/0)	+5.6 (8; 7/0)	+6.7 (8; 8/0)	+8.2 (10; 10/0)	+9.5 (10; 10/0)	+3.8 (10; 8/0)	+4.1 (10; 10/0)
	UD−D	+3.8 (7; 6/2)	+2.2 (8; 8/1)	+1.9 (7; 5/3)	+1.6 (7; 5/3)	+3.3 (6; 6/4)	+2.2 (6; 6/2)	+3.0 (6; 6/4)	+1.3 (7; 6/1)
$\mathcal{N}_{MLP}$	D−off	−0.6 (4; 4/5)	+0.8 (6; 5/4)	+3.4 (9; 9/1)	+2.7 (9; 8/1)	+4.8 (8; 7/1)	+5.1 (9; 9/0)	+2.8 (7; 6/2)	+2.9 (7; 7/2)
	UD−off	+1.2 (10; 8/0)	+1.2 (9; 8/0)	+5.1 (10; 10/0)	+4.9 (10; 10/0)	+7.7 (10; 10/0)	+7.3 (10; 10/0)	+5.8 (10; 10/0)	+4.6 (10; 10/0)
	UD−D	+1.6 (6; 6/4)	+0.4 (6; 5/3)	+1.3 (6; 6/3)	+1.9 (6; 6/2)	+3.5 (6; 6/4)	+2.5 (6; 6/4)	+3.1 (6; 6/3)	+1.3 (7; 7/3)

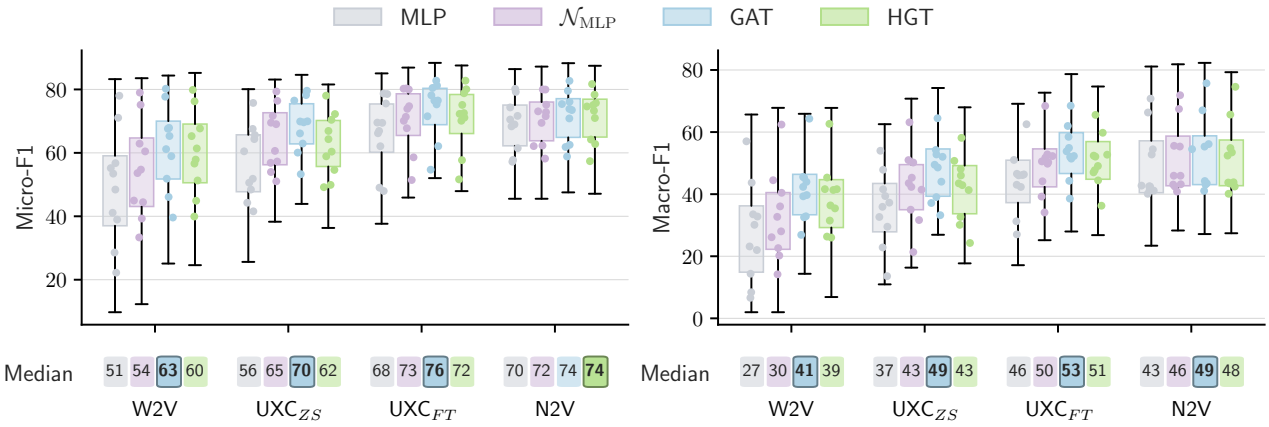


Fig. 4: Micro-F1 (left) and macro-F1 (right) of the MLP, $\mathcal{N}_{MLP}$, GAT, and HGT with undirected neighbors, by input representation. Each point is the median of one system over 100 runs; boxes show the spread across systems, and the values below are the median of medians across systems, with the highest in each group in bold.

predictor choice and aggregation direction affect the contribution of dependency neighbors. This section discusses these findings together.

Dependency Information in File Representations

Dependency information can support mapping before message passing, through the graph position encoded by Node2Vec or through dependency-guided adaptation of a code encoder. The advantage of these representations over Word2Vec, and of fine-tuned UniXcoder over its zero-shot counterpart, suggests that dependency information is useful beyond what the evaluated lexical and pretrained code representations provide. The representations also differ in training, vector size, and source content, so these comparisons are between representation pipelines and not between isolated information sources.

The usefulness of this signal varies with the relation between dependency structure and the reference architecture. Node2Vec's micro-F1 is lower on the three least homophilic systems, SH-3D, HDF, and T.Mates (Table 2), with a median of 58 compared with 72 on the remaining seven. Node2Vec learns dependency proximity from files that co-occur on random walks, without using the GT mapping. When dependencies frequently cross module boundaries, files from different modules may co-occur on walks and receive similar vectors, making the modules harder to distinguish.

The advantage of fine-tuned UniXcoder over Node2Vec on SH-3D and T.Mates suggests that code content may provide complementary evidence in these mixed neighborhoods. HDF does not follow this pattern, so low homophily alone does not explain their relative performance; HDF is also the smallest system, and every representation performs comparatively poorly on it. Homophily is used here to interpret the observed pattern, not to select a representation, since computing it requires the reference mapping.

Encoding dependencies in the initial representation and aggregating dependency neighbors during mapping are two uses of the same information source, and the second does not necessarily replace the first. Word2Vec and zero-shot UniXcoder gain more from neighbors than Node2Vec and fine-tuned UniXcoder do, yet fine-tuned UniXcoder still has the highest median macro-F1 for every predictor. Conversely, useful dependency information in a representation does not guarantee a benefit from further aggregation: RQ3 shows that this benefit depends on both aggregation direction and predictor choice.

Folder Organization and Architectural Exceptions

The gains from adding folder nodes in RQ2 show that directory organization helps reproduce the reference mapping. They do not show that this information must reach a file through message passing over folder nodes, since supplying the same information directly in the file vector matches or exceeds the accuracy of folder nodes on most systems. Extending a directory assignment is also different from identifying files that depart from it. Comparing FM, FFM, and FD on the directory baseline's errors helps assess this distinction (Section 5.2.2), but does not isolate architectural exceptions from errors caused by missing local supervision.

Lucene and HDF have pure directories under the reference mapping, so their baseline errors do not reflect conflicts between directory and module membership. They may instead arise from directories without labeled examples, although the table does not establish this. FFM recovers most baseline errors on Lucene and half on HDF, but only about a fifth on A.UML, whose directories are also pure. Directory purity therefore does not ensure that folder nodes recover baseline errors.

Bash and HDC have mixed directories. On baseline errors, FFM is similar to FM on Bash and less accurate on HDC. JabRef further illustrates why overall gains are insufficient: adding folder context raises overall accuracy while lowering accuracy on baseline errors. These comparisons do not establish whether the affected files depart from their directory's majority module.

The directory rule serves as a reference point for interpreting the learned models. Where directory organization closely follows the architecture, it already reproduces most of the mapping. On most evaluated systems, the added modeling complexity therefore brings little accuracy over the rule; where directories are less reliable, content and dependencies could be complementary signals, which these systems cannot show. Evaluating the additional value of a learned mapper therefore requires separating files in directories without labeled examples from files whose module differs from their directory's majority module. The present analysis does not establish how reliably the compared models handle the latter. A risk is that a model given folder information reproduces the directory assignment, including its errors. The low accuracy on baseline errors on most systems is consistent with this, but we did not test it.

RQ2 uses HGT to model the node and relation types of the heterogeneous variants. Although HGT gains the least from neighbor access in RQ3, its median macro-F1 gap to GAT is only 1.2 to 2.4 points, far smaller than the folder gains in RQ2. This suggests that predictor choice alone is unlikely to explain those gains, but whether they persist with GAT on the heterogeneous variants remains untested.

Dependency Neighbors and Aggregation Direction

The benefit of dependency neighbors depends on how the predictor aggregates them. HGT gains least, while GAT achieves the highest median macro-F1 with every representation under undirected aggregation, although differences between predictors are small relative to variation across systems. GAT and HGT differ in their attention formulations and parameterizations, so the comparison does not isolate why GAT performs better. These findings concern the evaluated file-level predictors, not the folder- and member-aware configurations or the usefulness of dependency types in general.

The preferred aggregation direction appears to depend more on the system than on the predictor or representation, although the pattern is not uniform across metrics. This suggests that neither direction is consistently preferable across systems. Agreement across configurations and models describes this consistency rather than providing independent evidence, since the comparisons share systems and splits.

Undirected aggregation adds reverse edges, so the comparison does not isolate propagation direction from neighborhood size. A reversed-edges-only setting would help distinguish these effects and remains for future work. With Node2Vec, directed aggregation lowers median macro-F1 for all three predictors. Since its vectors were learned on an undirected graph, this may reflect a mismatch between the neighborhoods encoded during initialization and those used during prediction, but this explanation remains untested.

Connectivity and Neighbor Information

Connectivity describes how much neighboring context is available, measured here by average node degree, while homophily describes whether linked files share a module. Few neighbors limit the context available to shallow message passing, whereas Node2Vec can encode more distant context through random walks. Limited connectivity may therefore constrain the benefit of additional aggregation even when the initial structural representation remains useful.

Several results are consistent with this interpretation. With Node2Vec and undirected aggregation, no predictor changes significantly on Bash and HDC, the two lowest-degree systems. In FM and FFM, dependencies between member nodes require two layers to reach file predictions; we

therefore examined whether the macro-F1 change from one to two layers relates to average degree. Exploratory Kendall's τ analyses [29] show positive associations in FFM across all three representations, but in FM mainly with Word2Vec.

Connectivity may also help explain the effect of aggregation direction. T.Mates and SH-3D combine low homophily with high average degree, gain most from file-level message passing with Node2Vec in RQ2, and favor directed aggregation in RQ3. Bash and HDC favor undirected aggregation, as does HDF, which has low homophily but much lower average degree than T.Mates and SH-3D. Reverse edges may supply missing context in sparse neighborhoods but introduce additional cross-module context in dense, low-homophily neighborhoods. This mechanism remains untested: the ten-system comparison cannot separate connectivity from homophily, system size, or label scarcity.

Implications for Evaluation

Evaluation should establish what the input representation already provides, then isolate access to neighborhood information within a fixed predictor. Comparing a GNN model with an MLP changes both the predictor and its access to dependency structure; enabling and disabling neighborhood information within the same predictor tests whether that access helps. The same applies to folder information: a comparison of graph variants should include a model that receives that information directly.

Micro-F1 and macro-F1 should be read together, since correctly mapping more files does not necessarily improve performance across modules. For tools intended to identify architectural exceptions, aggregate scores should also be accompanied by performance on files whose module differs from the majority module of their directory. These comparisons help distinguish the contributions of representations and neighbor processing from gains that mainly reproduce directory organization.

The intended use also matters. If mapping supports code reorganization, the existing directory structure may be something to reconsider rather than reproduce. Strong performance based on that structure does not establish which information would remain useful when the organization changes. The present study does not test this setting, but motivates evaluating information sources against the intended task rather than treating any representation or predictor as generally preferable.

7 Related Work

Architecture recovery and code-to-architecture mapping both infer a system's modular structure from implementation artifacts, such as static dependencies, code semantics, and directory organization. Recovery produces a decomposition without a reference architecture, while mapping assigns entities to the modules of a given one. We evaluate mapping and draw on recovery research for representations of dependencies, lexical content, and project organization.

Recovery techniques have historically been built around one dominant signal: dependency graphs, partitioned by search-based optimization [32, 36, 50, 33, 18] or clustering [41, 42, 1, 61, 38, 51], identifiers and comments [14, 11, 55, 28], or file names and paths [2]. Each signal is noisy in isolation, since shared utilities introduce cross-module edges and reused terminology mixes entities from many modules, so hybrid methods combine sources [35, 53, 37, 58, 52, 65, 48, 66].

Mapping began with orphan adoption [62] and the attraction functions of HuGMe [7, 8], and later varied the signal and the learner: textual similarity [4], supervised classifiers [3], and semi-supervised Naive Bayes combining lexical and dependency-derived information [44, 46]. Text-based approaches compare classifiers for module assignment [12], recommend mappings from natural-language architecture descriptions [59], or integrate package- and class-level granularity [60]. More recent work uses TF-IDF, GloVe, and LLM representations [27], extracts architectural concepts from documentation and code [13], or prompts LLMs to map implementation units to a reference architecture, which faces context-size limits and repeated processing costs on large codebases [57]. Johansson and Caporuscio map functions and files to textual module descriptions with pre-trained embeddings and find files classified more reliably than functions, with performance associated with vocabulary overlap and class separability [26].

Representation-learning work on recovery encoded one or two sources at a time. NEGAR clusters Node2Vec embeddings of the dependency graph alone [6]. Jabeen et al. [24] study dependencies alone and with folder organization, comparing random walk-based methods and variational graph autoencoders. GAER [23], a graph autoencoder, and ARL [25], a contrastively trained GAT, each fix folder and identifier tokens into one encoding without comparing alternatives. For mapping, GNNs were evaluated on homogeneous and dependency-type-aware graphs without folder or member nodes and without separating graph contribution from feature contribution [21]. Its extension combines Word2Vec identifier embeddings and binary bag-of-words representations of folder path tokens with dependencies in one model [22].

These approaches motivate our distinction between representing an information source and using it during prediction. Text-based mapping reports weaker results for functions than for files [26], but does not examine whether member information helps as context for the file. Our comparisons span lexical, structural, and contextual inputs but retain differences in their training procedures and feature content, so the matched dependency controls and directory-error analyses

assess specific design choices and do not turn the broader comparison into an isolated ranking of information sources.

8 Threats to Validity

Generalizability: We evaluate ten open-source systems in Java, C, and C++, each with one reference architecture. The findings may not extend to other systems or alternative architectural decompositions. The study is transductive: representation learning and graph processing can use unlabeled test nodes and their edges. It therefore does not establish performance on newly arriving files outside the observed graph or on maintaining a mapping as the system evolves. Other information sources, including change history and dynamic dependencies, are outside the evaluated scope.

Dependencies and reference mappings: Dependencies are extracted using static analysis, which may miss or misresolve dependencies. Preprocessing and build configuration can also affect the dependencies available for analysis. Because predictors that use neighbors depend on the extracted graph, such errors can affect their predictions. Models with neighbors enabled and disabled are trained separately. Their comparison therefore measures the effect of training with access to neighbor information, rather than removing neighbors from a fixed trained model.

The analyzed source code may not match the version a published mapping describes. Several benchmark mappings were created years before the source versions used here were obtained, and the files included in a system can depend on its build configuration, so the analyzed file set may differ from the one the mapping covers. In older systems, architectural drift may also weaken the correspondence between a file and its assigned module.

Files can plausibly belong to more than one module, and annotators may assign them differently. Because the initial labeled set is sampled from the same mapping used for evaluation, an incorrect assignment can both mislead the model during training and penalize an architecturally plausible prediction.

Experimental variation: We use 100 runs per configuration. These runs reuse the same systems and can include the same files, so they are not independent samples. With this many runs, small differences can reach statistical significance, so we report results per system and include effect sizes to distinguish the magnitude of a difference from its significance. The comparisons were not fixed in advance, so the analyses are exploratory and may favor patterns specific to this benchmark.

The runs use different initial labeled sets, but the contribution of stochastic model training to the observed variation was not assessed separately. Input representations were generated once per system and reused across runs, so variation arising from their generation was not assessed. The seeds governing model initialization and training were not retained, limiting exact reproducibility. Cliff's δ treats the paired scores as two independent groups, so it describes the separation of the two score distributions and not the paired differences.

Interpretation of results: Performance measures agreement with the reference architecture, not whether a prediction is the only architecturally reasonable assignment. Folder organization may reflect the same decisions used to define that architecture, so strong directory-based predictions do not by themselves demonstrate recovery of architectural intent. The directory-error analysis applies only where the directory baseline makes errors, which excludes C.Img, SH-3D, and T.Mates in most or all splits, and it does not isolate files whose module differs from the majority module of their directory. Message-passing depth was varied only for HGT in RQ2, with one and two layers, so the results do not show how deeper models or other predictors respond. It reports accuracy only, so it does not show how performance differs between small and large modules.

Relationships between performance and structural properties are reported as associations, not causal effects, since the benchmark does not allow these properties to be separated from one another or from system size and label availability. The fine-tuning gain is that of the whole dependency-guided procedure, since no alternative adaptation objective was evaluated, so it is not attributed to dependency topology alone.

9 Conclusion

This study examines how code text, static dependencies, folder organization, and member-level information contribute to code-to-architecture mapping on ten systems. The findings show that mapping performance depends not only on which information is available but also on how it is represented and used.

Representations that encode dependency information perform better than the evaluated lexical and pretrained code representations. Node2Vec and dependency-guided fine-tuned UniXcoder outperform Word2Vec on most systems, and fine-tuning improves UniXcoder over its zero-shot counterpart on nearly all systems. However, neither Node2Vec nor fine-tuned UniXcoder is consistently best across systems.

Folder nodes produce the largest and most consistent gains within the HGT variants, but a directory rule using the same labeled files reaches similar accuracy on most systems, and folder paths supplied as file features match or exceed folder nodes on most systems. These gains do not consistently translate into better predictions on files the rule assigns incorrectly. Member nodes provide no consistent improvement with one or two HGT layers, and their effects cannot

generally be separated from changes in feature construction and placement. These structural findings are specific to HGT; whether they persist with other predictors remains untested, particularly given HGT's smaller gains from dependency neighbors in the file-level comparison.

Keeping the input representations fixed while enabling and disabling access to dependency-neighborhood information shows that neighbor aggregation can complement representations that already encode dependencies. Undirected aggregation raises the median scores for all three predictors with every representation. GAT achieves the highest median macro-F1, while HGT gains the least. The better aggregation direction varies between systems, although the experiments do not isolate propagation direction from neighborhood size. Evaluation should therefore distinguish information already encoded in the representations from the contribution of neighborhood access. The findings support assessing these contributions for the intended mapping task, rather than treating any representation or predictor as generally preferable.

Future work. Future research could separate which aspects of graph context account for the observed differences. Undirected aggregation changed both propagation direction and neighborhood size, and member nodes changed both graph structure and feature placement, so comparing incoming, outgoing, and combined neighborhoods at matched sizes, and adding member nodes while preserving each file's own features, would show which change matters. Another direction is configuration selection on unfamiliar systems. Homophily, which helped interpret the system differences, requires reference module assignments, so selection rules based on properties observable in the dependency graph alone could be developed on some systems and evaluated on others. Finally, a model trained on one version could be evaluated on new and changed files in later versions without retraining, to test whether the learned information supports mapping as systems evolve.

References

1. Andritsos P, Tsaparas P, Miller RJ, Sevcik KC (2004) Limbo: Scalable clustering of categorical data. In: International Conference on Extending Database Technology, Springer, pp 123–146, DOI 10.1007/978-3-540-24741-8_9
2. Anquetil N, Lethbridge TC (1999) Recovering software architecture from the names of source files. Journal of Software Maintenance: Research and Practice 11(3):201–221, DOI 10.1002/(sici)1096-908x(199905/06)11:3<201::aid-smr192>3.0.co;2-1
3. Bibi M, Maqbool O, Kanwal J (2016) Supervised learning for orphan adoption problem in software architecture recovery. Malaysian Journal of Computer Science 29(4):287–313
4. Bittencourt RA, Santos GJdS, Guerrero DDS, Murphy GC (2010) Improving automated mapping in reflexion models using information retrieval techniques. In: 17th Working Conference on Reverse Engineering, pp 163–172
5. Brunsfeld M, contributors (2018) Tree-sitter: An incremental parsing system for programming tools. `https://tree-sitter.github.io/tree-sitter/`, accessed: 2026-04-22
6. Chen J, Wang Z, Jiang Y, Pang J, Zhang T, Pan M, Sun J (2022) NEGAR: network embedding guided architecture recovery for software systems. In: 2022 29th Asia-Pacific Software Engineering Conference (APSEC), IEEE, pp 367–376, DOI 10.1109/apsec57359.2022.00049
7. Christl A, Koschke R, Storey MA (2005) Equipping the reflexion method with automated clustering. In: 12th Working Conference on Reverse Engineering (WCRE'05), IEEE, pp 10–pp
8. Christl A, Koschke R, Storey MA (2007) Automated clustering to support the reflexion method. Information and Software Technology 49(3):255–274
9. Clements P, Garlan D, Little R, Nord R, Stafford J (2003) Documenting software architectures: views and beyond. In: 25th International Conference on Software Engineering, IEEE, pp 740–741, DOI 10.1109/icse.2003.1201264
10. Cliff N (1993) Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological bulletin 114(3):494, DOI 10.1037/0033-2909.114.3.494
11. Corazza A, Di Martino S, Maggio V, Scanniello G (2011) Investigating the use of lexical information for software system clustering. In: 2011 15th European Conference on Software Maintenance and Reengineering, IEEE, pp 35–44, DOI 10.1109/csmr.2011.8
12. Florean A, Jalal L, Sinkala ZT, Herold S (2021) A comparison of machine learning-based text classifiers for mapping source code to architectural modules. In: 15th European Conference on Software Architecture
13. Fuchß D, Liu H, Hey T, Keim J, Koziolek A (2025) Enabling architecture traceability by llm-based architecture component name extraction. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA), IEEE, pp 1–12
14. Garcia J, Popescu D, Mattmann C, Medvidovic N, Cai Y (2011) Enhancing architectural recovery using concerns. In: 2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011), IEEE, pp 552–555, DOI 10.1109/ase.2011.6100123
15. Garcia J, Krka I, Mattmann C, Medvidovic N (2013) Obtaining ground-truth software architectures. In: 35th International Conference on Software Engineering (ICSE), IEEE, pp 901–910, DOI 10.1109/icse.2013.6606639
16. Grover A, Leskovec J (2016) node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining, pp 855–864, DOI 10.1145/2939672.2939754
17. Guo D, Lu S, Duan N, Wang Y, Zhou M, Yin J (2022) Unixcoder: Unified cross-modal pre-training for code representation. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 7212–7225
18. Harman M, Yao X (2010) Software module clustering as a multi-objective search problem. IEEE Transactions on Software Engineering 37(2):264–282, DOI 10.1109/tse.2010.26
19. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 770–778
20. Hu Z, Dong Y, Wang K, Sun Y (2020) Heterogeneous graph transformer. In: Proceedings of the Web Conference 2020, pp 2704–2710
21. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2025) Graph convolution networks for mapping source code entities to architectural modules. In: 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), IEEE, pp 472–481, DOI 10.1109/icsa-c65153.2025.00071

22. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2026) Automatic code-to-architecture mapping with graph neural networks, under review
23. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2026) GAER: Graph auto-encoders for unsupervised software architecture recovery. Proceedings of the ACM on Software Engineering 3(FSE), DOI 10.1145/3808187
24. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2026) Graph embeddings for software architecture recovery. Nordic Machine Intelligence DOI 10.5617/nmi.13153, URL https://journals.uio.no/NMI/article/view/13153
25. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2026) Graph representation learning for software architecture recovery. In: ESANN
26. Johansson N, Caporuscio M (2025) Mapping source code to software architecture using embedding models: Insights from an analysis of data properties. SSRN Electronic Journal URL https://ssrn.com/abstract=5798927, preprint, SSRN 5798927
27. Johansson N, Caporuscio M, Olsson T (2024) Mapping source code to software architecture by leveraging large language models. In: European Conference on Software Architecture, Springer, pp 133–149, DOI 10.1007/978-3-031-70797-1_8
28. Kargar M, Isazadeh A, Izadkhah H (2017) Semantic-based software clustering using hill climbing. In: 2017 International Symposium on Computer Science and Software Engineering Conference (CSSE), IEEE, pp 55–60, DOI 10.1109/csicsse.2017.8320117
29. Kendall MG (1938) A new measure of rank correlation. Biometrika 30(1-2):81–93
30. Lenhard J, Blom M, Herold S (2019) Exploring the suitability of source code metrics for indicating architectural inconsistencies. Software Quality Journal 27(1):241–274
31. Lutellier T, Chollak D, Garcia J, Tan L, Rayside D, Medvidović N, Kroeger R (2017) Measuring the impact of code dependencies on software architecture recovery techniques. IEEE Transactions on Software Engineering 44(2):159–181, DOI 10.1109/tse.2017.2671865
32. Mancoridis S, Mitchell BS, Chen Y, Gansner ER (1999) Bunch: A clustering tool for the recovery and maintenance of software system structures. In: Proceedings IEEE International Conference on Software Maintenance (ICSM'99), IEEE, pp 50–59, DOI 10.1109/icsm.1999.792498
33. Maqbool O, Babri HA (2004) The weighted combined algorithm: A linkage algorithm for software clustering. In: Eighth European Conference on Software Maintenance and Reengineering (CSMR 2004), IEEE, pp 15–24, DOI 10.1109/csmr.2004.1281402
34. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. Advances in Neural Information Processing Systems 26
35. Misra J, Annervaz K, Kaulgud V, Sengupta S, Titus G (2012) Software clustering: Unifying syntactic and semantic features. In: 2012 19th Working Conference on Reverse Engineering, IEEE, pp 113–122, DOI 10.1109/wcre.2012.21
36. Mitchell BS, Mancoridis S (2006) On the automatic modularization of software systems using the bunch tool. IEEE Transactions on Software Engineering 32(3):193–208, DOI 10.1109/tse.2006.31
37. Mkaouer W, Kessentini M, Shaout A, Koligheu P, Bechikh S, Deb K, Ouni A (2015) Many-objective software remodularization using nsga-iii. ACM Transactions on Software Engineering and Methodology 24(3):1–45, DOI 10.1145/2729974
38. Mohammadi S, Izadkhah H (2019) A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code. Information and Software Technology 105:252–256, DOI 10.1016/j.infsof.2018.09.001
39. Murphy GC, Notkin D, Sullivan K (1995) Software reflexion models: Bridging the gap between source and high-level models. In: 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering, pp 18–28
40. Nair V, Hinton GE (2010) Rectified linear units improve restricted boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning (ICML), Omnipress, pp 807–814, URL https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf
41. Naseem R, Maqbool O, Muhammad S (2013) Cooperative clustering for software modularization. Journal of Systems and Software 86(8):2045–2062, DOI 10.1016/j.jss.2013.03.080
42. Naseem R, Deris MM, Maqbool O, Shahzad S (2019) Euclidean space based hierarchical clusterers combinations: an application to software clustering. Cluster Computing 22(Suppl 3):7287–7311, DOI 10.1007/s10586-017-1408-0
43. Nguyen HV (2023) Depends: Multilingual dependency analysis tool. https://github.com/multilang-depends/depends, accessed: 2025-08-17
44. Olsson T, Ericsson M, Wingkvist A (2019) Semi-automatic mapping of source code using naive bayes. In: Proceedings of the 13th European Conference on Software Architecture, pp 209–216
45. Olsson T, Ericsson M, Wingkvist A (2021) s4rdm3x: A tool suite to explore code to architecture mapping techniques. Journal of Open Source Software 6(58)
46. Olsson T, Ericsson M, Wingkvist A (2022) To automatically map source code entities to architectural modules with naive bayes. Journal of Systems and Software 183:111095, DOI 10.1016/j.jss.2021.111095
47. van den Oord A, Li Y, Vinyals O (2018) Representation learning with contrastive predictive coding. arXiv preprint arXiv:180703748
48. Papachristou M (2019) Software clusterings with vector semantics and the call graph. In: Proceedings of the 27th ACM Joint ESEC/FSE, pp 1184–1186, DOI 10.1145/3338906.3342483
49. Parnas DL (1972) On the criteria to be used in decomposing systems into modules. Communications of the ACM 15(12):1053–1058
50. Parsa S, Bushehrian O (2005) A new encoding scheme and a framework to investigate genetic clustering algorithms. Journal of Research and Practice in Information Technology 37(1):127–143
51. Pourasghar B, Izadkhah H, Isazadeh A, Lotfi S (2021) A graph-based clustering algorithm for software systems modularization. Information and Software Technology 133:106469, DOI 10.1016/j.infsof.2020.106469
52. Puchala SPR, Chhabra JK, Rathee A (2022) Ensemble clustering based approach for software architecture recovery. International Journal of Information Technology 14(4):2013–2019, DOI 10.1007/s41870-021-00846-0
53. Rathee A, Chhabra JK (2017) Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering. In: Advanced Informatics for Computing Research: ICAICR 2017, Springer, pp 94–106, DOI 10.1007/978-981-10-5780-9_9
54. Richards M, Ford N (2025) Fundamentals of Software Architecture: A Modern Engineering Approach. O'Reilly Media
55. Risi M, Scanniello G, Tortora G (2012) Using fold-in and fold-out in the architecture recovery of software systems. Formal Aspects of Computing 24(3):307–330, DOI 10.1007/s00165-011-0199-y
56. Romano J, Kromrey JD, Coraggio J, Skowronek J (2006) Appropriate statistics for ordinal level data: Should we really be using t-test and cohen'sd for evaluating group differences on the nsse and other surveys. In: annual meeting of the Florida Association of Institutional Research, vol 177
57. Rukmono SA, Ochoa L, Chaudron M (2024) Deductive software architecture recovery via chain-of-thought prompting. In: 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp 92–96, DOI 10.1145/3639476.3639776
58. Sadat Jalali N, Izadkhah H, Lotfi S (2019) Multi-objective search-based software modularization: Structural and non-structural fea-

tures. Soft Computing 23(21), DOI 10.1007/s00500-018-3666-z
59. Sinkala ZT, Herold S (2021) Inmap: Automated interactive code-to-architecture mapping recommendations. In: 18th International Conference on Software Architecture, pp 173–183
60. Sinkala ZT, Herold S (2023) An integrated approach to package and class code-to-architecture mapping using inmap. In: 2023 IEEE 20th International Conference on Software Architecture (ICSA), IEEE, pp 164–174
61. Teymourian N, Izadkhah H, Isazadeh A (2020) A fast clustering algorithm for modularization of large-scale software systems. IEEE Transactions on Software Engineering 48(4):1451–1462, DOI 10.1109/tse.2020.3022212
62. Tzerpos V, Holt RC (1997) The orphan adoption problem in architecture maintenance. In: Working Conference on Reverse Engineering, IEEE, pp 76–82
63. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y (2017) Graph attention networks. arXiv preprint arXiv:171010903 DOI 10.48550/arXiv.1710.10903
64. Wilcoxon F (1945) Individual comparisons by ranking methods. Biometrics bulletin 1(6):80–83, DOI 10.2307/3001968
65. Yang K, Wang J, Fang Z, Wu P, Song Z (2022) Enhancing software modularization via semantic outliers filtration and label propagation. Information and Software Technology 145:106818, DOI 10.1016/j.infsof.2021.106818
66. Zhang Y, Xu Z, Liu C, Chen H, Sun J, Qiu D, Liu Y (2023) Software architecture recovery with information fusion. In: Proceedings of the 31st ACM Joint ESEC/FSE, pp 1535–1547, DOI 10.1145/3611643.3616285